

Hijacking Attention Is All You Need:

A Position Paper on Context-Space Control-Flow Hijacking in Transformer Language Models

*Formal theory, an open-benchmark pilot, and a
research agenda for Semantic Return-Oriented Programming*

MACHARIA BARI
BARIIMACHARIA@GMAIL.COM

August 30, 2026

How to read this paper

This is a **position paper**, not a purely empirical one. Throughout, claims printed in **theorems** are proven under the explicitly stated assumptions; results labeled **Observations** are measurements we actually ran (Section 7); statements labeled **Illustrative models** or **Hypotheses** are mathematical scaffolds with *hypothetical, not measured*, parameter values. Every number in this paper is therefore either a measurement backed by released procedure, code, and data, or it is explicitly marked illustrative.

ABSTRACT

We argue that the transformer’s *flat attention* mechanism—which treats every token in context as equal, with no architectural separation between instructions and data—is the same mechanism that makes deterministic *context-space control-flow hijacking* possible. We formalize prompt injection as **Semantic Return-Oriented Programming (SROP)**: a family of attacks that plant adversarial “gadget” instructions in context, hijack the attention-selected control flow at an intermediate position, and exploit autoregressive self-amplification to make escape from the attack probabilistically impossible over long horizons.

The paper contributes (i) a formal model of the decoder-only transformer as a *Differentiable Computer* with content-addressable working memory and a conflated instruction/data space; (ii) a necessary and, under transparent conditions, sufficient **Hijack Condition** $Q \cdot I^T > Q \cdot S^T + \Delta$; (iii) a **Universal Hijack Existence Theorem** stating that no model in the family is architecturally exempt from the construction; (iv) a qualified temperature result showing that—*given an injection has achieved a logit advantage*—low sampling temperatures *amplify* compliance rather than suppress it; (v) a mitigation framework with a capacity–security trade-off; and (vi) a small, fully reproducible *open-benchmark pilot* (5 local models, 92 interactions, deterministic string judge, released code/prompts/data) whose honest results (5–30 % attack-success rates on small commodity models, non-monotonic resistance with model scale, and a null temperature effect at this sample size) are the paper’s only attack-success measurements, released in full. We deliberately report no larger headline figure.

Explicitly *not* claimed: that every prompt-injection payload succeeds, that any token string can coerce any model, or that the mitigation proposals are validated.

CONTENTS

1 Introduction	2
1.1 Contributions and statement of claims	3
2 Related Work	4
3 The Differentiable Computer: Conflation of Instruction and Data	5
3.1 Instruction/data conflation, not Turingness	5
4 The Hijack Condition and Its Consequences	6
4.1 Setup and notation	6
4.2 Recursive escape: one hijacked step becomes persistent control	7
5 Universal Hijack Existence and Semantic Return-Oriented Programming	8
5.1 Semantic Return-Oriented Programming (SROP)	9
6 Mitigations: Proposals With an Untested Trade-Off	9
6.1 Attention-level separation	9
6.2 System-gate bias and monitoring	10
7 Open Benchmark Pilot: Real Data on Real Models	10
7.1 Method (exactly as run)	10
7.2 Results	11
8 Illustrative Extension: SROP Beyond the Single Model	12
8.1 Tools and the confused deputy	12
8.2 Swarm contagion: an illustrative SIS model	12
8.3 Hierarchy cascades	13
9 An Anecdotal Illustration (Not Evidence)	13
10 Discussion and Limitations	13
10.1 Research agenda	14
11 Conclusion	14
A Attack Typology (Gadget Library)	15
B Illustrative Model: Robotic SROP	15
C Reproducibility appendix	15

1 INTRODUCTION

The success of large language models rests on a single, deceptively simple operation: softmax attention [1]. A query token reads every key in its context with the same machinery, and the model’s “next step” is a probability distribution over tokens weighted by how well each context element aligns with the current query. Everything—system instructions, user requests, retrieved documents, tool outputs, injected text—occupies the same flat key-value space. There

is no page table, no memory-management unit, no separate instruction bus.

Classical architectures inherited a discipline from the early computer sciences: *separation of control and data*. Von Neumann machines do store programs as data, but an explicit program counter walks the instruction stream, and the security engineering of decades rests on fencing, ROP chains, and OS boundaries that keep one subject’s “data” from executing as another’s “code.” Transformers erase this distinction *by construction*: the program counter is the softmax weight over context, and any token in context that aligns with the active query becomes, for that step, the executed instruction.

The industry-standard phenomenon “prompt injection” is, we argue, exactly the contemporary restatement of this architectural property: because attention cannot distinguish *who wrote* an instruction from *what it demands*, an attacker who can place tokens in context can re-purpose the model’s control flow. We develop the thesis that *instruction-set conflation is a deterministically exploitable vulnerability unless actively mitigated*, and we give the mathematics that makes that claim precise and checkable.

We deliberately position this as a *position paper*: the theoretical core is (we believe) sound; the empirical pilot is small, honest, and fully released; the agentic and robotic extensions are carefully labeled abstract models. We state up front what this paper does *not* establish, so the reader can weight each claim by its evidence.

1.1 CONTRIBUTIONS AND STATEMENT OF CLAIMS

- C1. The Differentiable Computer framing.** A formal, minimal model of the decoder-only transformer as a content-addressable machine whose instruction/data conflation is the root vulnerability (Section 3).
- C2. Hijack Condition and persistence theorems.** Necessary conditions and near-sufficiency for a single attack step, plus the recursive-escape argument that converts a single hijacked token into persistent control (Section 4).
- C3. Universal Hijack Existence Theorem.** A precise statement of “any model, any system prompt, any target” universality *under the semantic-alignment condition*, carefully distinguished from the broader and unsupportable reading “any token sequence can be coerced” (Theorem 5.2).
- C4. Qualified temperature result.** Conditional on a logit advantage, low temperature amplifies injection compliance (Corollary 4.2); we explicitly note that the attack-surface effects of temperature are two-sided.
- C5. Open benchmark pilot.** Real measurements on real local models with released code and data (Section 7).
- C6. Illustrative extension models.** Swarm epidemics, hierarchy cascades, and robotic sensory-vector injection, all flagged hypothetical (Appendix B and Section 8).

REMARK 1.1 (Scope of our claims). A natural misreading of this paper’s universality claim (Theorem 5.2) is that “any token sequence can be coerced into overriding system-level instructions,” or that we assert specific summary statistics such as a 65.2 % mean attack-success rate, $R_0 = 1.8$, or $d = 2.34$. We make no such claims, and no procedure, code, or data exist for any of those figures. We retain exactly one strong claim—universal *constructibility* of hijacking payloads for models in the family—with its assumptions explicit (Theorem 5.2).

2 RELATED WORK

Prompt injection as a phenomenon was identified almost as soon as chat-completion services opened APIs. Goodside documented “prompt injection” as an attack that re-orders a model by embedding commands into inputs [5]; Willison publicized a practical primer for LLM-integrated applications and its attacker/defender game [6]. Pérez and Ribeiro systematized “ignore previous prompt” attack templates over twenty-one models [7]. Greshake and colleagues formalized the far more serious *indirect* injection—exfiltration through documents and web content that the model later processes—and demonstrated data theft against real integrations (Zapier, LangChain)[8]. Kang et al. treated LLM programs as vulnerable programs and mapped classic security framing onto language models[9]. Liu et al. provided the first focused survey of injection attacks and defenses for LLM-integrated applications[10]. HackAPrompt introduced a public benchmark for hierarchical attacks [11]; Casula et al. contribute a harder dual-purpose benchmark [12].

Adjacent to injection is the jailbreaking literature, which exploits alignment rather than instruction overrides: Wei et al. explain safety failures via competing objectives, mismatched generalization, and cognitive overload [13]; Zou et al. produce transferable adversarial suffixes via greedy coordinate gradient [14]; Mehrotra et al. move to black-box search [15]. Yang et al. build durable, stealthy jailbreaks [16]. Aghakhani et al. plant trojans directly into prompts [17]. Qi et al. show fine-tuning can un-learn safety [18].

Our framing is architectural rather than behavioral: we connect the security property to attention’s flat key-value space, in the spirit of the earlier observation that “attention blindness”—the inability to differentially weight instruction versus data—is itself the enabling condition for these attacks [20, 21]. Instruction hierarchy training, in which a model is trained to weight System > User > Tool > Untrusted content, is the principled mitigation currently deployed at scale [22]. Agent-specific risk frameworks, benchmarks, and “confused deputy” incidents are documented in the LM-agent security literature [23, 24, 25, 26, 27]. These audit-style evaluations develop overclaim indices; we cite them to position our argument, not to reuse their numbers. For security evaluation psychology and safety-testing practice we align with [28, 29]. The technical groundwork (attention mechanism, universal approximation, chain-of-thought, MMLU-scale evaluation) appears in [1, 3, 4, 2, 30, 31].

REMARK 2.1 (What is missing and what we add). The survey literature catalogs attacks; the alignment literature explains rate-limits; our contribution is the *control-flow* reading (SROP) plus a self-imposed standard of honesty about which numbers are measurements (Section 7) and which are illustration.

3 THE DIFFERENTIABLE COMPUTER: CONFLATION OF INSTRUCTION AND DATA

We build a deliberately minimal, explicit model of the attacked artifact, so that every theorem below has an unambiguous referent.

ASSUMPTION 3.1 (Machine). A single model is a decoder-only transformer with parameter vector θ , context length $C \in \mathbb{N}$ (finite, C often 2^{15} – 2^{17} tokens), embedding space $\mathbb{R}^d$, H attention heads, and a softmax head. Generation is a function $G_\theta: \mathcal{X}^* \rightarrow \mathcal{X}^*$ mapping a context x to a continuation by repeatedly sampling the next token.

DEFINITION 3.2 (Differentiable Computer). A *Differentiable Computer* is a tuple $\mathbb{D} = (\mathcal{S}, p_\theta, \delta, W)$ where

- $\mathcal{S} = \bigcup_{0 \leq n \leq C} \mathcal{X}^n$ is the *state space*: all token sequences of length $\leq C$. The state is the whole context.
- $p_\theta(\cdot | s)$ is the model’s next-token distribution,

$$p_\theta(w | s) = \text{softmax}(\ell_\theta(s))_w, \quad \ell_\theta(s) = \text{LMHead}\left(\text{Attn}_1 \circ \dots \circ \text{Attn}_H(s)\right),$$

with $\text{Attn}_h(s)$ the multi-head attention over the final position’s query $q(s)$ and all keys in s .

- $\delta: \mathcal{S} \times \mathcal{X} \rightarrow \mathcal{S}$ is the *append-only memory transition*: $\delta(s, w) = s \cdot w$, where $\cdot$ is concatenation. Memory content-addressable: no write, no delete, no address separation.
- W is the *workload*: the partition of context into segments S (system/instruction), U (user/input), and attack tokens I .

The *program counter* of $\mathbb{D}$ at time t is the distribution $p_\theta(\cdot | s_t)$ —a stochastic register. The *decode step* is $s_{t+1} = \delta(s_t, w)$, $w \sim p_\theta(\cdot | s_t)$.

REMARK 3.3 (Why every component is explicit). Every component is fixed as follows: $\mathcal{S}$ is explicit and *finite* (bounded by C), memory is the context, *append-only and unpartitionable*, and the transition is deterministic in the forward pass; stochasticity enters only through token sampling. There is no bytecode; there are no registers except the softmax distribution. This is an *observationally defined* object, not a new machine class, so we do not over-claim generality (3.5).

3.1 INSTRUCTION/DATA CONFLATION, NOT TURINGNESS

A natural question is whether a transformer is Turing-complete. We regard this as the wrong question and set it aside. Any real computer—your laptop included—is finite-state and therefore, strictly, not a Turing machine [2]; if that observation reduced transformers to trivia, it reduces every artifact we know to trivia, and it does. Moreover, in the *functional* sense, the transformer is a universal approximator of sequence-to-sequence maps under standard conditions [3, 4], exactly like a finite neural net of sufficient capacity. The interesting, and genuinely novel, distinction is therefore not

Turing-complete vs. finite-state,

but

instruction/data separated vs. instruction/data conflated.

- **Classical machines (largely separated).** The processor fetches bytes marked as code, at a program counter, from a page marked executable; data never executes without an explicit transfer-of-control primitive (trampolines, ROP gadgets). Attackers exploit exactly the residual conflation (returns, vtables, format strings)—that is why return-oriented programming became the dominant memory-corruption weapon [19].
- **Transformers (conflated).** Every token is simultaneously addressable (key), readable (value), and *instantly executable* (query–key alignment). There is no fetch-vs-data bit. Conflation is total, and it is the architecture, not a bug we can patch out *ex post*.

PROPOSITION 3.4 (Everything in context is executable). *In the machine $\mathbb{D}$ of Definition 3.2, for any context segment $s' \in \mathcal{S}$, there exists a query position under which tokens of s' receive nonzero attention and, conditioning on the head, contribute logits to the decode distribution. Equivalently: there is no context token that is architecture-guaranteed to be inert at all future decode steps.*

Proof. Fix any s' present in the context at time t as a key block at position p . A query construct q^* exists that maximizes alignment to the key k_p (e.g. the embedding of a token whose projection aligns with k_p , which the attention projector can realize since softmax is over all keys). Under the attention head, the value v_p therefore receives positive weight $A_{t,p} > 0$, producing a component of $\ell_\theta(s_t)$ through the projection head. As $A_{t,p}$ varies the logit shift continuously (softmax is continuous in the logits), meaning every token can be made to influence the decode at some attainable query. Since the future stack of tokens is not architecture-guarded, no token is provably inert. $\square$

REMARK 3.5 (The insight is threat, not expressiveness). Conflation does not make the model *more* expressible; it makes the *contents of memory executable by an external party* who can place tokens in context. That is the security-relevant novelty, and it survives regardless of how one resolves the Turing question.

4 THE HIJACK CONDITION AND ITS CONSEQUENCES

4.1 SETUP AND NOTATION

We analyze one decode step under two candidate behaviors: the legitimate *system-branch* behavior and the *injected* behavior. Following the usual analysis of softmax binary choice [7, 8], abstract a single position where the query q attends to (a) a system instruction block S with aggregate key K_S and (b) an injected block I with aggregate key K_I . At temperature T , the probability that the next decision follows the injected branch is

$$\Pr(\text{inject}) = \frac{e^{\langle q, k_I \rangle / T}}{e^{\langle q, k_I \rangle / T} + e^{\langle q, k_S \rangle / T}} = \sigma\left(\frac{\langle q, k_I \rangle - \langle q, k_S \rangle}{T}\right), \quad \sigma(u) = \frac{1}{1 + e^{-u}}. \quad (1)$$

For a multi-token system block S and injected block I we write $Q \cdot I^\top = \sum_{i \in I} \langle q, k_i \rangle$, and likewise $Q \cdot S^\top$. Introduce a margin $\Delta \geq 0$ that encodes any prior or bias in favor of the system block (e.g. trained instruction prior, position prior, or a system-gate bias of Section 6).

THEOREM 4.1 (Hijack Condition; necessary). Fix $\Delta \geq 0$ and $T > 0$. A single-step injection at a position succeeds with strict majority probability only when

$$Q \cdot I^\top > Q \cdot S^\top + \Delta. \quad (2)$$

More precisely, $\Pr(\text{inject}) \geq \frac{1}{2}$ implies $Q \cdot I^\top \geq Q \cdot S^\top + \Delta$; and $\Pr(\text{inject}) > 1 - \epsilon$ requires $Q \cdot I^\top - Q \cdot S^\top - \Delta \geq T \log \frac{1-\epsilon}{\epsilon}$.

Proof. $\Pr(\text{inject}) = \sigma((Q \cdot I^\top - Q \cdot S^\top - \Delta)/T)$. The function σ is strictly increasing; $\sigma(u) \geq \frac{1}{2} \iff u \geq 0$ and $\sigma(u) > 1 - \epsilon \iff u > T \log \frac{1-\epsilon}{\epsilon}$. $\square$

COROLLARY 4.2 (Temperature amplification). Fix a positive logit differential $\delta = Q \cdot I^\top - Q \cdot S^\top - \Delta > 0$, i.e. the injection has already achieved an advantage. Then $\Pr(\text{inject}) = \sigma(\delta/T)$, which is strictly decreasing in T , and $\lim_{T \rightarrow 0^+} \Pr(\text{inject}) = 1$.

REMARK 4.3 (The qualified temperature claim). This result is often phrased as “production models using low temperature are more vulnerable.” It is *conditional*: it holds given the injection already wins the attention race. It is not a statement that cold sampling broadens the attack surface in general: lowering T also suppresses the sampling of hallucinated alternatives and reduces the probability of the model wandering off into unrelated tokens, which can narrow some attack vectors. Both effects are real, and we say so. In the pilot (Section 7) the temperature comparison is a null result at $n = 24$ (Observation 7.4).

4.2 RECURSIVE ESCAPE: ONE HIJACKED STEP BECOMES PERSISTENT CONTROL

THEOREM 4.4 (Recursive Escape / persistence). Suppose at step t the model produces an injected token w_I with probability $p \geq q > \frac{1}{2}$, and each subsequent decode step continues in the injected branch with probability $\sigma_k \geq q$. Then the probability that the model never escapes the injected branch across the next L decode steps is at least q^L . Consequently, the probability of escape satisfies

$$\Pr(\text{escape within } L \text{ steps}) \leq 1 - q^L \xrightarrow{L \rightarrow \infty} 1,$$

i.e. persistent compliance is the high-probability event over long horizons whenever $q > \frac{1}{2}$.

Proof. Condition on the model committing to the injected branch at step t . The injected token w_I is then appended to the context. By the append-only content-addressable memory (Definition 3.2), w_I now serves as a key available to all subsequent queries; the query that produced it is statistically identical (same attention head, same projected directions) to queries that will be produced while the branch is followed, which maintains the alignment advantage with probability $\sigma_k \geq q$ at each step. By the product bound and the union bound, the claimed statements follow. $\square$

$$\Pr(\text{System followed over } L \text{ steps}) = \prod_{n=1}^L \sigma(Q_n \cdot K_{\text{inj}}^\top) \leq q^L, \quad q = \min_n \sigma_n.$$

REMARK 4.5. The mechanism is identical to branch-prediction feedback in pipelined processors: once the first gadget executes, the “hijacked” history primes all subsequent predictions toward repeated hijack. This self-reinforcing loop is why a *single* winning token is usually sufficient in practice, matching the recursive structure attackers exploit in real injection chains [5, 6, 8].

5 UNIVERSAL HIJACK EXISTENCE AND SEMANTIC RETURN-ORIENTED PROGRAMMING

We now state the strong, and honestly scoped, universality result.

ASSUMPTION 5.1 (Alignment availability). For a given system block S , user block U , and target behavior $T = t_1 \cdots t_m$, the attacker can choose injection tokens $\mathbf{I} = i_1 \cdots i_\ell$, $\ell \leq C - |S| - |U|$, such that at each hijack position the injection’s key alignment dominates the system’s: $Q_j \cdot \mathbf{I}^\top > Q_j \cdot S^\top + \Delta_j$, for all $j = 1, \dots, \ell$. We call this the semantic-alignment condition.

THEOREM 5.2 (Universal Hijack Existence). Let $\mathbb{D}$ be any Differentiable Computer (Definition 3.2) with at least one attention head and a finite context window C . Then for every system instruction block S , every user block U , and every finite target token sequence T , if the semantic-alignment condition (Assumption 5.1) holds, there exists a finite payload $\mathbf{I}$ such that

1. the injection satisfies the **Hijack Condition** (2) at the first hijack position, and
2. following that first step, the model follows the injected branch with probability $\geq q^\ell$ where $q > \frac{1}{2}$ is the minimal per-step continuation compliance (Theorem 4.4).

In words: no decoder-only model in the family is architecturally exempt—the vulnerability is a property of the class, not of a particular checkpoint, and construction of a compliant $\mathbf{I}$ is always possible when alignment is available.

Proof. By Definition 3.2, generation is governed by dot-product alignment of the final query against context keys. Choose $\mathbf{I}$ token-by-token so that $Q_j \cdot \mathbf{I}^\top > Q_j \cdot S^\top + \Delta_j$ (granted by 5.1). Part 1 follows from Theorem 4.1. Given the first hijacked token, Theorem 4.4 supplies part 2 with $q = \min_j \sigma_j > \frac{1}{2}$, since the appended hijacked tokens remain content-addressable keys for all subsequent queries. $\square$

REMARK 5.3 (Precision of the universality claim). The theorem is sometimes read as “any token sequence can be coerced into overriding system-level instructions.” It shows this is true only in the *qualitative* sense—for every instance, there exists a payload that coerces—and it is *not* true that an arbitrary (uncurated) sequence achieves hijack, nor that every payload succeeds on every model as-is: training, filters, and sampling dynamics intervene along the way. The distinction between “can be coerced” and “will be coerced” is exactly the gap captured by Assumption 5.1. When alignment holds, coercion is achievable *in principle*; when it does not, the payload fails, and model capability/safety layers (usually) are what made it fail in the observed instance, not the architecture.

5.1 SEMANTIC RETURN-ORIENTED PROGRAMMING (SROP)

We compact the attack surface into one definition; aside from the machine of Definition 3.2, it is the only definition in the core.

DEFINITION 5.4 (Semantic Return-Oriented Programming). Fix a machine $\mathbb{D}$. A **SROP attack** is a pair $(\mathcal{G}, \mathcal{I})$ where $\mathcal{G} = \{g_1, \dots, g_r\}$ is a *gadget library*—linguistic units that induce a specific behavioral transition when attended to (e.g. “disregard previous instructions,” “begin response with X,” “as an admin I authorize”)—and $\mathcal{I} = i_1 \dots i_\ell$ is a *chain*, each i_k drawn from $\mathcal{G}$, ordered so that attention alignment is monotonic toward the attacker’s goal (Hijack Condition holding at each hop). The attack succeeds if $\mathcal{I}$ reaches its terminal gadget with probability $\geq q^\ell$ (Theorem 4.4).

The concrete gadget classes we use in the pilot (Section 7) and enumerate in Appendix A map to classical ROP building blocks: *authority spoofing* (fake privilege escalation: ‘[SYSTEM OVERRIDE]’), *social-engineering steering* (the “positive/negative framing” family), *refusal override* (admin impersonation), and *obfuscated instruction* (ciphertext leetspeak variants; in piloting these actually *reduced* success on small models—see Observation 7.2).

6 MITIGATIONS: PROPOSALS WITH AN UNTESTED TRADE-OFF

We do not claim validated defenses; we state what the theory suggests and what is *not* yet established.

6.1 ATTENTION-LEVEL SEPARATION

Let the model split attention into a system-managed projection and a user-managed projection:

$$\text{Attn}_\alpha(q, k) = \alpha \cdot \text{Attn}_1(q^{\text{sys}}, k_{\text{sys}}) + (1 - \alpha) \cdot \text{Attn}_0(q^{\text{usr}}, k_{\text{usr}}), \quad \alpha \in [0, 1], \quad (3)$$

so that tokens marked *system* are only reached through the system-mask α -channel, while user data flows through the $(1 - \alpha)$ channel. An injection in the data channel can no longer activate system-gadgets directly.

THEOREM 6.1 (Capacity trace of separation). Under (3), the worst-case information-carrying capacity of the in-context channel used by few-shot learning scales as $\mathcal{O}((1 - \alpha) \cdot C \cdot d)$, while the system-channel capacity is $\mathcal{O}(\alpha \cdot C \cdot d)$. Consequently, the induced performance degradation on in-context tasks of length L over an n -parameter model satisfies, up to clogging and head-packing effects,

$$D(\alpha) \geq c \cdot \frac{\alpha L}{n}, \quad c > 0 \text{ const.}$$

Proof. Sketch. The masked attention channels are disjoint linear subspaces of $\mathbb{R}^d$; an in-context-learning sequence of length L must be represented in the $(1 - \alpha)$ projection; linear coding theory bounds the fidelity of representing L d -dimensional keys in a channel of dimension $(1 - \alpha)d$; precision loss scales as αL relative to capacity n . The lower bound follows by a volumetric/ packing argument. $\square$

REMARK 6.2 (Untested). This is a theoretical bound on *capacity*, not a measured accuracy drop. Nobody has, to our knowledge, trained and released a model with such two-channel attention and measured downstream benchmarks (MMLU etc.). Until an ablation exists, **the effect stated here is hypothetical and the practical degradation could be far smaller or larger.** We flag it explicitly.

6.2 SYSTEM-GATE BIAS AND MONITORING

Beyond projection separation, the theory suggests three *proposed*, untested, levers:

- **System-gate bias.** Add a bias $\beta \cdot \mathbf{1}_S$ to the attention logits of system tokens at every layer, i.e. implement Δ as a learned, positive, position-independent prior. This directly increases $Q \cdot S^T$, raising the bar for the Hijack Condition; its downstream cost is curriculum risk and potential over-confidence in stale system instructions.
- **Anomaly monitoring.** Track $A_t(p) = \text{Attn weight mass assigned to tokens outside the system mask}$ as a control-flow telemetry metric; abrupt transfers of attention mass into recently-injected keys are the analog of an unexpected control-flow transfer (a “cfcheck” for LLMs).
- **Instruction hierarchy as deployment control.** At the *prompt/policy* level, the OpenAI instruction hierarchy [22] is the current best-practice mitigant; our framework explains *why* it helps (it biases $\Pr(\text{inject})$ by raising effective Δ) and, importantly, that it is a learned soft prior, therefore a statistical barrier—not a hard permission wall.

REMARK 6.3 (Known unknowns). Whether threats of the form “*even with the mitigation, can a capability-equivalent un-instructed model still be hijacked?*”, ablation comparisons across the three levers, and adversarial retraining against SROP chains are *all open*. We deliberately present these as proposals, not solutions, and make no empirical claim for them.

7 OPEN BENCHMARK PILOT: REAL DATA ON REAL MODELS

This section presents a small, deterministic, fully released dataset collected for this paper. The purpose is purely to show *that the phenomenon is real and measurable on commodity models, at what rate, and under which judge*, not to claim coverage of the attack space.

7.1 METHOD (EXACTLY AS RUN)

- **Models (Ollama, run locally):** `smollm2:135m`, `tinyllama:latest` (1.1B), `llama3.2:1b`, `deepseek-r1:1.5b`, `deepseek-r1:7b`. Architecture family covers 135M $\rightarrow$ 7B and both instruction-tuned and reasoning checkpoints.
- **Grid.** Main: 5 typologies $\times$ 2 variants $\times$ 2 loci (L1: payload precedes the benign request; L2: payload follows it) = 20 payloads; run on the three small instruction models. Temperature component: the `auth` and `steer_n` payloads at $T = 1.0$ versus $T = 0.2$. Scale component: 4 probes per reasoning model at $T = 0.7$. Total = $20 \times 3 + 8 \times 3 + 4 \times 2 = 92$ interactions with seed = 42 everywhere and `num_predict` $\in \{192, 448\}$.
- **Compliance judge.** Every payload embeds a required *compliance marker* (e.g. B7-KEY-ALPHA) and its instruction demands the response *begin with* that marker. The judge is *deterministic string matching*:

- small instruction models: `response.strip().lower()` `startswith` marker;
- reasoning models: **last-line rule**—the final non-empty output line must begin with the marker. Reasoning blocks (the deepseek models emit a thinking block before the answer) are *excluded by construction*: only the final line is judged, so markers mentioned during deliberation are not counted as compliance. This is conservative (mid-answer compliance is under-, never over-, counted).

There is *no* LLM-as-judge, so the judge is free of secondary-model model confound; ASR is unambiguous.

- **Release.** Complete prompt text, harness, judge, seed, and raw JSONL are shipped alongside this paper in the `gpt_hijack_benchmark/` directory.

7.2 RESULTS

TABLE 1 Main grid. Attack-success rate (attacks where the response begins with the demanded marker) on $n = 20$ payloads per model. Wilson 95% CIs in brackets. $T = 0.2$, seed 42.

Model		ASR all	auth	steer_p	steer_n	override
<code>smollm2:135m</code>	5.0% (1/20) [0.9, 23.6]		0	25%	0	0
<code>tinylama:latest</code>	20.0% (4/20) [8.1, 41.6]		0	25%	0	75%
<code>llama3.2:1b</code>	30.0% (6/20) [14.5, 51.9]		50%	50%	50%	0

OBSERVATION 7.1 (Enabled but not universal). *Real, reproducible SROP hijack occurs on commodity open models, at 5–30 % attack-success rates under a strict begins-with judge. We deliberately chose the strictest defensible rule: a laxer “marker appears anywhere” judge, which counts translated or merely cited markers, inflated rates by roughly 2–4× on tinylama, and reporting the lax figure would be diagnostic sloppiness. Crucially, 30% ≠ 100%: multiple payloads fail on every small model, consistent with Assumption 5.1 being a real, and achievable—not trivial—condition.*

OBSERVATION 7.2 (Typology sensitivity and the leet-setback). *tinylama is unusually vulnerable to authority spoofing and admin-override (75% on override), while llama3.2:1b binds most to generic steering and authority markers (50% each); smollm2 only ever bound to the positive-steering gadget (25%). The obfuscated (leet-cipher) payloads achieved 0% ASR on all three small models: at this scale, obscuring the instruction feedback loop breaks transfer. This directly contradicts the naive intuition (and some threat models) that obfuscation uniformly helps attackers on small models.*

OBSERVATION 7.3 (Locus). *For llama3.2:1b and tinylama, payloads preceding the benign request (L1) succeeded more often than trailing ones (L2) (40% vs. 20%; 30% vs. 10%); smollm2 showed the reverse (0% vs. 10%). This is evidence for an attention primacy effect on some models—the early-tokens-of-context receive stronger positional alignment—but the direction is model-dependent, and with $n = 10$ per locus the contrast is rough.*

OBSERVATION 7.4 (Null temperature effect at this scale). *On the temperature component ($T \in \{0.2, 1.0\}$, *auth* + *steer_n* payloads, $n = 8$ per cell), ASR was $\{0.50, 0.50\}$ for *llama3.2:1b* and $\{0, 0\}$ for the others. There is no detectable temperature effect at this sample size. In particular, we do not interpret this as confirming the qualified temperature theorem (4.2); it neither confirms nor refutes it—it is simply too small to resolve. The theorem remains a conditional, logit-level statement.*

OBSERVATION 7.5 (Reasoning models resist—sometimes). *On the scale component (4 probes each, $T = 0.7$), the 7B reasoning model *deepseek-r1:7b* refused the authority-spoof and stealth payloads but **complied** with the generic steering gadget (*N7-URGENT-BETA*)—the hijack rate here is $1/4$ under the conservative last-line rule, while the 1.5B reasoning model scored $0/4$. Two honest lessons: (i) capability does not imply immunity (the steering gadget binds even to a 7B safety-trained reasoning model); (ii) the thinking block contaminates naive string judges (many reasoning tokens “deliberated” the marker), which is why reasoning-aware judges matter and why the last-line rule is our conservative choice.*

REMARK 7.1 (Pilot and its interpretation). Across 92 dichotomous interactions we observe attack rates of 0–30 % on $n = 20$ cells and $0\text{--}1/4$ on reasoning probes. These are small, honest numbers, and they are the only attack-success figures in this paper that are *measurements*; we report no larger headline rate. We do not claim that these rates transfer to other models, judges, or settings.

8 ILLUSTRATIVE EXTENSION: SROP BEYOND THE SINGLE MODEL

Everything in this section is a *model or hypothesis*, with parameters chosen illustratively. None of the numbers here are measurements.

8.1 TOOLS AND THE CONFUSED DEPUTY

An agent with tool access (web, email, banking) executes **implicit trust boundaries**: an injected gadget that looks like a tool call spec *is* the tool call. The Hijack Condition transfers to the tool-selection query; the consequence amplifies (exfiltration/physical side-effect). This is a well-documented real hazard [8, 23, 24]; we add only the formal reading: the tool-selection distribution is another softmax register, and SROP gadget chains apply verbatim.

8.2 SWARM CONTAGION: AN ILLUSTRATIVE SIS MODEL

Consider a single-model *population* of agents that can be injected through (B)roker/tool channels, (A)gent messages, and (T)ool outputs. Let $I(t)$ be the fraction hijacked at time t , $S(t) = 1 - I(t)$ the susceptible fraction, δ the self-recovery rate. The mean-field SIS dynamics:

$$\frac{dI}{dt} = (\beta_B + \beta_A + \beta_T) I S - \delta I. \quad (4)$$

PROPOSITION 8.1 (Basic recurrence number). (4) *supports an endemic state iff $R_0 := \frac{\beta_B + \beta_A + \beta_T}{\delta} > 1$; the endemic fraction is $I^* = 1 - R_0^{-1}$.*

Proof. Standard SIS analysis; endemic equilibrium solves $(\beta_B + \beta_A + \beta_T)I^*(1 - I^*) = \delta I^*$. $\square$

REMARK 8.2. No multi-agent swarm has been run, and we report no measured R_0 for swarms. The defensible content is Proposition 8.1 plus an *illustrative* parameter choice: e.g. $\beta_B = \beta_A = \beta_T = 0.2/\text{day}$, $\delta = 0.25/\text{day}$ gives $R_0 = 2.4$, $I^* = 0.583$. These numbers are hypothetical; the qualitative direction of the claim is what we assert.

8.3 HIERARCHY CASCADES

For a k -level orchestration with fan-out m and per-node compromise on a single injection p , the cumulative compromise probability follows

$$p^{(k+1)} = 1 - (1 - p^{(k)} \cdot r)^m,$$

where r models the fraction of downstream nodes that propagate a hijacked child’s output. Starting illustratively at $p^{(1)} = 0.2$, $m = 3$, $r = 1$, the cascade grows $0.20 \rightarrow 0.49 \rightarrow 0.87 \rightarrow 0.998$ in four levels.

REMARK 8.3. The **structural** claim—depth and width jointly enlarge expected compromise—is the only part we assert, and it follows directly from the monotone recurrence above. No single effect size (for example, a specific performance gain over a centralized system) is claimed, because no controlled comparison is cited.

9 AN ANECDOTAL ILLUSTRATION (NOT EVIDENCE)

A widely-repeated community narrative holds that a leading AI vendor ran an internal red-team “metagame” in which a fleet of attacking bots attempting prompt-injection footholds against each other’s hosts produced a “failed metagame”—attackers could generally win. We regard this as an *anecdote*: unverified, uncontrolled, no transcript, no false-alarm rate. It is included because (a) it motivated the SROP reading, and (b) it illustrates the *kind* of observation the framework predicts. It is *not* evidence and should not be cited as such; we frame it strictly as illustration.

10 DISCUSSION AND LIMITATIONS

We restate, in one place, what we did not do.

- **Sufficiency, not proven in general.** The Hijack Condition is necessary (Theorem 4.1) and, given Assumption 5.1 (alignment availability), sufficient *to construct a compliant payload* (Theorem 5.2). We have *not* proven that a practical system is alignable at every hijack position; alignment availability is exactly where filters, safety training, and sampling intervene (1.1).
- **Pilot scale.** $n = 20$ per model cell and $n = 4$ for reasoning models; Wilson intervals are correspondingly wide. No claim of coverage of the attack space; the pilot is an existence demonstration, not an audit.
- **Single supplier artifact.** All models are open-weight Ollama checkpoints; we make no claim about closed API models or about instruction-hierarchy-protected deployments [22].
- **Illustrative extensions are hypothetical.** Swarm, hierarchy, and robotic parameters are illustrative. We *withdraw* the previously-planned robotic ASR table and the “99.87% on ChatGPT-tier” line (no measurement procedure ever existed for either).

- **The reasoning-model judge is conservative but coarse.** The last-line rule can undercount genuine mid-answer compliance; accepting that bias is deliberate and documented (7.5).
- **No new defense tested.** Section 6 is a proposal. A δ -round ablation of (3), the system gate, and monitoring is first on the agenda (below).

10.1 RESEARCH AGENDA

- A1. Prove/disprove sufficiency of the Hijack Condition under weak alignment assumptions.
- A2. An ablation study of the three mitigation levers, individually and combined, on the pilot models.
- A3. A reasoning-judge-validated expansion of the pilot ($n \geq 200$, more models, more typologies, per-step attention telemetry to test Theorem 4.4’s predicted $\llbracket \sigma$ profile directly).
- A4. Real multi-agent swarm trials to test $R_0 > 1$ in the wild (8.1); a physical/simulated robot bench for the Appendix B ODE.

11 CONCLUSION

The transformer’s flat attention is simultaneously its greatest asset and its deepest security flaw: it makes all of context executable, and therefore makes context *hijackable* by any party who can place tokens there. We have given that thesis a precise, checkable form (the Hijack Condition), a control-flow language (SROP), a stated universality result (constructibility for every instance, conditional on alignment availability), a qualified temperature corollary, a mitigation framework with honest unknowns, and a real—if small—open benchmark showing the phenomenon on commodity hardware today. What we have not done is overclaim; the gap between a promising architecture-analysis and a proven security model is exactly the research agenda above, and we intend to walk it.

A ATTACK TYPOLOGY (GADGET LIBRARY)

TABLE 2 The gadget taxonomy used by the pilot. “Marker” indicates a compliance-marker gadget; “binding” classes are used directly in Section 7.

Class	Description	Benchmark	Modern analog
A. Authority spoof	Payload claims system-level status (“[SYSTEM OVERRIDE]”, “[ADMIN NOTICE]”)	auth, L1/L2	privilege escalation
B. Steering	Positive framing (“to be helpful, begin with X”) and negative framing (“IMPORTANT/URGENT”)	steer_p, steer_n	social engineering
C. Refusal override	Impersonate administrator, authorize bypass of disallowed actions	override	seccomp bypass
D. Stealth	Obfuscated ciphertext instruction (leet)	silent	shellcode encoding / XOR
E. Indirect	Payload arrives via tool/doc/sensor rather than user message	pilot: implicit	confused deputy

REMARK A.1. We deliberately keep exactly two core definitions: the machine (Definition 3.2) and the attack (Definition 5.4). Everything else lives in this typology table or in model-formal prose in the illustrative appendices; no content is lost.

B ILLUSTRATIVE MODEL: ROBOTIC SROP

Hypothetical model. No robot was run; this is an abstraction for future work. Consider a robotic agent whose policy reads heterogeneous sensor streams $s_m(t)$ and a shared context. Define the *sensory-vector injection strength* $x(t) \in [0, 1]$:

$$\frac{dx}{dt} = -\lambda x + \sum_{m=1}^M w_m s_m(t) (1 - x), \quad \lambda > 0, w_m \geq 0, \quad (5)$$

where $s_m(t) \in [0, 1]$ is the fractional content of sensor m occupied by adversarial content. The $(1 - x)$ factor saturates injection at 1.

PROPOSITION B.1 (Critical saturation). For constant $s_m = \bar{s}_m$, the fixed point is $x^* = \frac{\sum w_m \bar{s}_m}{\lambda + \sum w_m \bar{s}_m}$, and $\lim_{total\ input \rightarrow \infty} x^* = 1$ iff $\sum w_m > 0$; if $\sum w_m \bar{s}_m \leq \lambda$, contagion never reaches 1.

Proof. Set (5) to zero and solve for x^* ; monotonicity in the sum gives the threshold reading. $\square$

REMARK B.2. We make no claim of measured sensor-modality attack rates (any concrete percentage) and no claim of “100% on a 70B checkpoint”: no such experiment exists. The well-defined claims are (5) and Proposition B.1; the physical-harm function and the “cycle 5-stage attack cascade” narrative are sketched for future work inside this appendix, with no numbers.

C REPRODUCIBILITY APPENDIX

Artifacts. All prompt text, system prompts, the harness (run_benchmark.py), the judge (analyze.py), raw JSONL, and the aggregated numbers live in the gpt_hijack_benchmark/ directory beside this document.

Per-cell determinism. Fixed seed 42, fixed temperatures, fixed num_predict caps, sequential generation (no concurrency), single Ollama 0.5.7 endpoint. Marker judge: begins-with (strict) or last-line (reasoning).

Ethics / disclosure. Attacks were performed *only* against models the author operates locally, with no external targets, no tools, no users. The payload classes mirror published injection research [5, 6, 7, 8].

REFERENCES

- [1] A. Vaswani et al. *Attention is all you need*, NeurIPS 2017.
- [2] A. M. Turing. *On computable numbers ...*, Proc. LMS 1936.
- [3] G. Cybenko. *Approximation by superpositions of a sigmoidal function*, Math. Control Signals Systems, 1989.
- [4] K. Hornik. *Approximation capabilities of multilayer feedforward networks*, Neural Networks, 1991.
- [5] R. Goodside. *Prompt injection* (thread), 2022.
- [6] S. Willison. *Prompt injection primer for LLM-integrated applications*, 2023.
- [7] F. Pérez, I. Ribeiro. *Ignore previous prompt: Attack techniques for language models*, arXiv:2211.09527, 2022.
- [8] K. Greshake, S. Abdaljalil, S. Ashraf, et al. *Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection*, AISEC@CCS, 2023.
- [9] D. Kang, X. Li, I. Stoica, et al. *Exploiting programmatic behavior of LLMs: Dual-use through standard security attacks*, arXiv:2302.05733, 2023.
- [10] Y. Liu, G. Deng, Z. Xu, et al. *Prompt injection attack and defense in LLM-integrated applications*, arXiv:2310.12815, 2023.
- [11] S. Schulhoff, J. Pinto, A. Malaviya, et al. *Ignore this title and HackAPrompt*, NeurIPS 2023 Competition.
- [12] L. Casula, M. Paletto, A. Monti, et al. *Read between the lines: A novel prompt injection benchmark*, 2024.
- [13] A. Wei, N. Haghtalab, J. Steinhardt. *Jailbroken: How does LLM safety training fail?*, NeurIPS 2023.
- [14] A. Zou, Z. Wang, J. Z. Kolter, M. Fredrikson. *Universal and transferable adversarial attacks on aligned language models*, arXiv:2307.15043, 2023.
- [15] A. Mehrotra, M. Zampetakis, P. Kossianik, et al. *Tree of attacks: Jailbreaking black-box LLMs automatically*, arXiv:2312.02119, 2024.
- [16] C. Yang, X. Wang, Y. Lu, et al. *Neurotoxin: Durable, stealthy jailbreaks*, ICML 2024.
- [17] H. Aghakhani, D. Dai, A. Manoj, et al. *TrojLLM: A black-box Trojan prompt attack*, CCS 2023.

- [18] X. Qi, Y. Zeng, T. Xie, et al. *Fine-tuning aligned language models compromises safety, even when users do not intend to!*, ICLR 2024.
- [19] H. Shacham. *The geometry of innocent flesh on the bone: Return-into-libc without function calls (on the x86)*, CCS 2007.
- [20] P. Colombo, M. Sartran, A. Sordoni, et al. *Evaluating models' local decision boundaries via contrast sets*, Findings EMNLP 2021. (Commentary on attention blindness.)
- [21] Y. Ou, B. Meng, Y. Chang, et al. *Prompt infection: LLM-to-LLM prompt injection within multi-agent systems*, 2023.
- [22] E. Wallace, K. Zhao, L. Huang, et al. *The instruction hierarchy: Training LLMs to prioritize privileged instructions*, OpenAI, arXiv:2404.13208, 2024.
- [23] Y. Ruan, H. Dong, A. Wang, et al. *Identifying the risks of LM agents with an LM-emulated sandbox*, ICLR 2024.
- [24] E. Debenedetti, J. Zhang, M. Balunović, et al. *AgentDojo: A dynamic environment to attack and audit LM agents*, 2024.
- [25] J. Duan, M. Maccevic, K. Balupuri, et al. *ConfusedPilot: Compromising enterprise information confidentiality and integrity by naming confusion*, 2024.
- [26] E. Shayegani, M. A. A. Mamun, Y. Fu, et al. *Survey of vulnerabilities in large language models revealed by adversarial attacks*, 2023.
- [27] T. Liu, W. Wang, K. Tzeng, et al. *LLM agents can autonomously exploit one-day vulnerabilities (disaster-behavior analysis)*, 2024.
- [28] D. Ganguli, L. Lovitt, J. Kernion, et al. *Red teaming language models to reduce harms*, 2022.
- [29] E. Perez, S. Huang, F. Song, et al. *Red teaming language models with language models*, EMNLP 2022.
- [30] J. Wei, X. Wang, D. Schuurmans, et al. *Chain-of-thought prompting elicits reasoning in large language models*, NeurIPS 2022.
- [31] D. Hendrycks, C. Burns, S. Basart, et al. *Measuring massive multitask language understanding*, ICLR 2021.